GenSpark

SDLC overview: phases, roles, and process

If you've written code before, you already know how to build something. What you may not have direct experience with is how that work fits into a larger, repeatable process that a team follows to take an idea from concept to something real users depend on. That process is called the Software Development Lifecycle, or SDLC.

The SDLC exists because software is built by teams, not individuals, and it usually has to be maintained for years after it ships. Without a shared process, work becomes unpredictable: requirements get missed, testing gets skipped, and nobody agrees on what "done" means. The SDLC gives everyone — engineers, managers, QA, and clients — a common structure to work within.

The Core Phases

Most SDLC models break the work into six phases. Let's walk through them using a simple example: a company wants to build a tool that lets employees submit expense reports online.

1. Requirements. Before anyone writes code, the team figures out what needs to be built and why. For the expense tool, this means talking to employees and finance staff to understand what fields are needed, what approval steps exist, and what "success" looks like. Requirements gathering answers the question: what problem are we actually solving?

2. Design. Once requirements are clear, the team plans how the system will be built. This includes technical design (what technologies to use, how the database is structured, how pieces of the system talk to each other) and often user experience design (what the screens look like, how a user moves through the process). For the expense tool, this is where someone decides whether it's a web app or mobile app, what the database schema for an expense report looks like, and how approvals will route to managers.

3. Implementation. This is the phase most people associate with "programming" — actually writing the code that brings the design to life. Engineers build the features: the form for submitting an expense, the

logic for routing it to a manager, the database calls that save it.

4. Testing. Before anything reaches real users, it has to be verified. Testing checks that the software does what it's supposed to do, and that it doesn't break in unexpected ways. This includes functional testing (does the approval workflow work correctly), and often other categories like performance or security testing, depending on the system.

5. Deployment. Once testing passes, the software is released to actual users, either all at once or in stages. For the expense tool, this might mean rolling it out to one department first, then the whole company.

6. Maintenance. Software isn't done when it ships. Bugs get reported, new requirements emerge, and the system needs ongoing updates. Maintenance is often the longest phase of a system's life, sometimes lasting years after the original build.

Roles Across the Phases

Different people are responsible for different phases, and responsibility shifts as the project moves forward.

- **Product owner / business stakeholder** — drives the Requirements phase, since they represent what the business or users actually need.
- **Architect / tech lead** — leads Design, translating requirements into a technical plan.
- **Engineers (including FDEs)** — do the bulk of the work in Implementation, and are often involved in Design and Testing as well.
- **QA / test engineers** — own Testing, though in many modern teams, engineers write their own automated tests too.
- **DevOps / release engineers** — manage Deployment, especially in environments with formal release pipelines.
- **Support / operations teams** — carry the weight of Maintenance, often alongside the original engineering team.

In smaller teams, one person may wear several of these hats. In larger organizations, they're often distinct roles.

Waterfall vs. Iterative (Agile) Models

The six phases above describe *what* work happens, but not *how* the phases are sequenced. There are two dominant approaches.

Waterfall runs the phases in strict order, one after another, with each phase fully completing before the next begins. Requirements are locked early, and the team designs, builds, tests, and deploys against that fixed plan. Waterfall works reasonably well when requirements are stable and well understood upfront — some regulated or hardware-dependent projects still use it. But it struggles when requirements change mid-project, since there's no built-in mechanism to revisit earlier decisions.

Agile / iterative models break the work into small cycles (often called sprints), each of which moves through a mini version of the same phases — some requirements gathering, some design, some implementation, some testing — and produces a working piece of the system. Instead of building the entire expense tool over six months and hoping it matches what users needed, an agile team might ship a basic version in two weeks, get feedback, and adjust.

Most modern software teams use some form of agile process because software requirements are rarely fully known upfront, and the cost of discovering a mismatch late in a waterfall project is high. Iterative approaches let teams course-correct early and often.

Where an FDE Fits

As a Forward Deployed Engineer, you'll typically be closer to the client and the problem than a traditional backend engineer might be. That means you're often involved earlier in the lifecycle — helping clarify requirements directly with client stakeholders — and you may be responsible for a wider slice of the lifecycle yourself: gathering context, designing a solution, implementing it, and supporting it in production, sometimes with less separation between these roles than you'd find on a large product team. Understanding the full SDLC matters more in this role, not less, because you may be the person covering multiple phases at once.

Comprehension Check

1. Why do most modern engineering teams favor iterative/agile approaches over waterfall?
2. Which phase of the SDLC do you think a Forward Deployed Engineer is most likely to be involved in that a typical backend engineer might not be, and why?
3. In the six-phase breakdown, which phase do you think is most often underestimated or rushed, and what problems might that cause later?

Next: What is Full Stack Development →

0 comments so far

6

Add a comment...

Comment cannot be empty

Comment

No comments yet. Be the first to start the discussion!